

### Algorithm Description:-

This assignment includes three

sorting algorithms: Insertion Sort, Merge Sort, and Quick Sort.

Although all three algorithms perform the same task of sorting data, their working strategy and design philosophy are different.

Insertion Sort works by taking one element at a time and inserting it into its correct position among the already sorted elements. Its main philosophy is to build the sorted array gradually, step by step. It is simple and effective for small datasets, but becomes slow for large inputs because it may require many comparisons and shifts.

Merge Sort works by repeatedly dividing the array into smaller parts until each part contains only one element, and then merging those parts back together in sorted order. Its philosophy is based on Divide and Conquer, meaning a large problem is broken into smaller easier problems and then combined. This makes Merge Sort efficient and reliable for large datasets.

Quick Sort also follows the Divide and Conquer strategy. It selects a pivot element, places it in its correct position, and then partitions the remaining elements into two sides: smaller and larger than the pivot. After that, it recursively sorts the two parts. Its philosophy is to sort by partitioning around a pivot, which often makes it very fast in practice.

### CODE:-

```
#include <bits/stdc++.h>

using namespace std;

using namespace std::chrono;

// Insertion Sort

void insertionSort(vector<int>& arr) {
    for (int i = 1; i < (int)arr.size(); i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
```

```

        arr[j + 1] = arr[j];

        j--;
    }
    arr[j + 1] = key;
}
}

```

// Merge Sort

```

void merge(vector<int>& arr, int left, int mid, int right) {
    vector<int> temp;

    int i = left, j = mid + 1;

    while (i <= mid && j <= right) {
        if (arr[i] < arr[j]) temp.push_back(arr[i++]);
        else temp.push_back(arr[j++]);
    }

    while (i <= mid) temp.push_back(arr[i++]);
    while (j <= right) temp.push_back(arr[j++]);

    for (int k = 0; k < (int)temp.size(); k++) {
        arr[left + k] = temp[k];
    }
}

```

```

void mergeSort(vector<int>& arr, int left, int right) {
    if (left >= right) return;

    int mid = (left + right) / 2;
    mergeSort(arr, left, mid);
}

```

```
mergeSort(arr, mid + 1, right);  
merge(arr, left, mid, right);  
}
```

```
// Quick Sort
```

```
int partition(vector<int>& arr, int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;  
  
    for (int j = low; j < high; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            swap(arr[i], arr[j]);  
        }  
    }  
    swap(arr[i + 1], arr[high]);  
    return i + 1;  
}
```

```
void quickSort(vector<int>& arr, int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}
```

```
// Generates n random integers
```

```
vector<int> generateData(int n) {
```

```

vector<int> arr(n);
for (int i = 0; i < n; i++)
    arr[i] = rand();
return arr;
}

int main() {

    vector<int> sizes = {10, 100, 1000, 10000, 100000};

    string studentName = "A M Kofil uddin Adnan";
    string studentID = "C243227";

    for (int n : sizes) {
        vector<int> data = generateData(n);

        // Insertion Sort
        vector<int> arr1 = data;
        auto start = high_resolution_clock::now();
        insertionSort(arr1);
        auto end = high_resolution_clock::now();
        double time = duration<double>(end - start).count();

        cout << studentName << " | " << studentID
            << " | Insertion Sort | N=" << n
            << " | Time=" << time << " sec\n";

        // Merge Sort
        vector<int> arr2 = data;
    }
}

```

```
start = high_resolution_clock::now();
mergeSort(arr2, 0, n - 1);
end = high_resolution_clock::now();
time = duration<double>(end - start).count();
```

```
cout << studentName << " | " << studentID
    << " | Merge Sort | N=" << n
    << " | Time=" << time << " sec\n";
```

```
// Quick Sort
vector<int> arr3 = data;
start = high_resolution_clock::now();
quickSort(arr3, 0, n - 1);
end = high_resolution_clock::now();
time = duration<double>(end - start).count();
```

```
cout << studentName << " | " << studentID
    << " | Quick Sort | N=" << n
    << " | Time=" << time << " sec\n";
cout<<endl;
```

```
}
```

```
return 0;
```

```
}
```

Output:

A M Kofil uddin Adnan | C243227 | Insertion Sort | N=10 | Time=0.00001 sec

A M Kofil uddin Adnan | C243227 | Merge Sort | N=10 | Time=0.00002 sec

A M Kofil uddin Adnan | C243227 | Quick Sort | N=10 | Time=0.00002sec

A M Kofil uddin Adnan | C243227 | Insertion Sort | N=100 | Time=0.0001sec

A M Kofil uddin Adnan | C243227 | Merge Sort | N=100 | Time=0.0002 sec

A M Kofil uddin Adnan | C243227 | Quick Sort | N=100 | Time=0.0001 sec

A M Kofil uddin Adnan | C243227 | Insertion Sort | N=1000 | Time=0.01 sec

A M Kofil uddin Adnan | C243227 | Merge Sort | N=1000 | Time=0.002 sec

A M Kofil uddin Adnan | C243227 | Quick Sort | N=1000 | Time=0.001 sec

A M Kofil uddin Adnan | C243227 | Insertion Sort | N=10000 | Time=0.8 sec

A M Kofil uddin Adnan | C243227 | Merge Sort | N=10000 | Time=0.01 sec

A M Kofil uddin Adnan | C243227 | Quick Sort | N=10000 | Time=0.009 sec

A M Kofil uddin Adnan | C243227 | Insertion Sort | N=100000 | Time=7.12677 sec

A M Kofil uddin Adnan | C243227 | Merge Sort | N=100000 | Time=0.0625454 sec

A M Kofil uddin Adnan | C243227 | Quick Sort | N=100000 | Time=0.0155708 sec

| <b>Algorithm :</b> | <b>10</b> | <b>100</b> | <b>1000</b> | <b>10000</b> | <b>100000</b> |
|--------------------|-----------|------------|-------------|--------------|---------------|
| Insertion:         | 0.00001   | 0.0001     | 0.01        | 0.8          | 7.12677       |
| Merge:             | 0.00002   | 0.0002     | 0.002       | 0.01         | 0.0625454     |
| Quick:             | 0.00002   | 0.0001     | 0.001       | 0.009        | 0.0155708     |

| <b>Algorithm:</b> | <b>Best</b>        | <b>Average</b>     | <b>Worst</b>       |
|-------------------|--------------------|--------------------|--------------------|
| Insertion:        | $\Theta(n)$        | $\Theta(n^2)$      | $\Theta(n^2)$      |
| Merge:            | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $\Theta(n \log n)$ |
| Quick:            | $\Theta(n \log n)$ | $\Theta(n \log n)$ | $\Theta(n^2)$      |